Graph Analytics Orchestrator

CS6855 Course Project — Group 2 Report

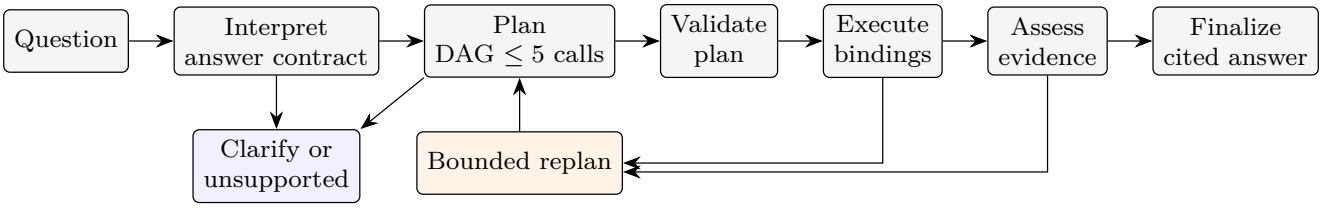
Pallepogu Rishi (CS23B048) Abhinand Shaji (ED22B044) Pervaje Surya Prasad Bhat (ED23B035)
Syed Sufiyan (NS26Z168) Mohd Hamza (PH22B016)

November 6, 2026

1 Overview

Our LangGraph orchestrator answers natural-language questions over five provided datasets (`airports`, `karate`, `lesmis`, `courses`, `projects`). It discovers a curated selection of Assignment 3 tools by reading their manifests rather than hard-coding query-to-tool mappings. Google Gemini supplies structured plans through its OpenAI-compatible API; `LLM_MODEL` configures the model and `LLM_API_KEY` supplies the key through the environment. The required entry point is `python run_queries.py --queries <file> --out answers.json`.

2 Design



1. **Curate and discover.** Offline tests check candidate manifests, schema-valid outputs, repeatability, independent numerical behavior and execution time. Selected tool files are copied unchanged into `registry/tools/`. Startup scans those manifests plus four instructor infrastructure tools; strict validation rejects missing or duplicate tools.
2. **Interpret.** A forced structured response produces an immutable answer contract: answer type, cardinality, ordering, required evidence and any genuine missing user choices. A dynamic graph catalogue supplies exact IDs and attributes. Bounded, question-relevant label hints help identify entities such as the instructor and administrator, but cannot themselves support a final answer.
3. **Plan and validate.** The planner receives the contract and the discovered tool catalogue. It creates an acyclic plan of at most five steps or returns a structured clarification/unsupported decision. Validator checks include tool-name membership, manifest arguments, dependencies and graph IDs.
4. **Execute and chain.** Ready steps run on one GraphStore. JSON Pointer bindings pass a prior `/result_graph_id` or resolved node ID into the next call. q19, for example, chains a Girvan–Newman split to PageRank on the larger registered subgraph. Tool outputs are validated before becoming evidence.
5. **Assess, finalize and recover.** Evidence assessment tests whether each contract requirement is supported and may request bounded replanning. The finalizer must return the required structured type and cite existing step IDs and output paths. JSONL traces record every LLM/tool call and final answer; retries, response caching, SQLite checkpoints and deterministic tool replay support interrupted batches.

3 Registry selection

The local curation report evaluated **186 candidates**; 167 passed its checks and one implementation was selected for each of **33 analytical tools**. Four infrastructure tools remain separate. The initial selection used the lowest median runtime among eligible candidates, with implementation ID as a tie-breaker. Because that choice is machine-dependent, the submitted `registry/tools/` must be checked against the implementation IDs recorded in `answers_public.json` and its traces, rather than rebuilt by speed alone. Some initial rejections were evaluator/output-shape false positives, not tool bugs.

4 Public-set evaluation

The public set contains 30 questions. We compare status, structured type and value against the supplied public expectations (0.1% tolerance for numbers; ranking order where relevant). The figures below are the *best trace-backed terminal results across recorded attempts*, not one uninterrupted run.

Expected class	Correct	Selected outcomes
Answerable, q01–q23	20/23	20 correct answers; q17 and q22 errored, q21 answered incorrectly.
Ambiguous, q24–q26	2/3	q24 and q25 clarified; q26 ended in error.
Unsupported, q27–q30	4/4	All four correctly returned unsupported.
Total	**26/30 (86.7%)**	21 answered (20 correct), 2 clarification, 4 unsupported, 3 error.

The q12 answer and trace agree on a Jaccard similarity of 0.432432, so q12 is counted. The q21 answer and trace both give c63, but that value comes from betweenness on the entire `lesmis` graph, not Valjean's circle; the expected local answer is c40. We make no hidden-set accuracy claim. Most successful answers use one or two tools within the five-call budget.

5 Analysis of remaining failures

1. **q17: derived-subgraph scope was misjudged.** An ego-network call registered the instructor's local graph; degree centrality on it ranked m1 first and m2 highest among m1's friends. Evidence assessment mistakenly treated the result as a ranking on the whole original graph. Further replanning hit the limit. The assessor must honor the executed `graph_id` and exclude the instructor when selecting a friend.
2. **q22: wrong analysis chain.** Javert's immediate circle requires node resolution, `ego_network`, and `motif_count` on the resulting graph. Initial plans embedded arguments inside tool names; later attempts chose clustering tools and exhausted replanning. The expected count for the full ego subgraph is 96 triangles; a local coefficient or triangles through Javert alone cannot substitute for that result.
3. **q21: wrong graph for the calculation.** Betweenness was calculated on all 77 `lesmis` characters, making Myriel (c63) rank second globally. The workflow then built Valjean's neighbour graph without recalculating betweenness there. Instead it should calculate betweenness on Valjean's 37-node ego graph (including Valjean), then exclude Valjean only when choosing the answer. That calculation ranks Javert (c40) next, at 0.062634.
4. **q26: ambiguity incorrectly pursued as a computation.** "How close" could mean hops, co-appearance strength or neighbourhood similarity. The later run tried weighted shortest-path steps and ended in error rather than requesting clarification. A shared-chapter weight represents strength, not a distance; interpreting it as path length changes the meaning of the question.

6 Registry implementation findings

The following were found in *unselected* candidates. We did not edit registry source.

1. **Major correctness bug: `motif_count.7cc0c2d7`.** It returns **52** triangles on `karate`; independent NetworkX counting and the selected implementation return **45**. A three-node induced subgraph gives a minimal reproducer:

   ```
   sub = store.subgraph("karate", ["m2", "m3", "m14"], "repro")   # one triangle
   tool.run(store, graph_id=sub)["result"]["triangle_count"]  # 2, expected 1
   ```

   The implementation orders two nodes by their positions in `g.nodes()` but filters the third using a string comparison of node IDs. These orderings differ for IDs such as `m14` and `m2`, causing double counting. It should use one consistent node order. No modification was made to the student tool.
2. **Smaller contract bug: `minimum_spanning_tree.5c98c8eb`.** A valid `lesmis`, `top_k=100` call serializes to **8,717 bytes**, above the required **8,192-byte** output ceiling. Its result selection limits item count, not serialized size.
3. **Minor limitation, not a wrong result: `maximal_cliques.ab9fb3a3`.** A valid `top_k=100` Karate request returns an explicit size-limit error rather than a bounded preview. It does not return an incorrect clique or violate the output-size limit.

Other candidates rejected by the initial harness were tested against their actual result shapes; false-positive rejections are not presented as bugs. Verified issues can be reported through the course's discussion channel without sharing source code.

7 Member contributions

- **Abhinand Shaji (ED22B044):** Curated and tested the registry tools. Improved executor replanning by reusing a unique node ID from prior evidence when the tool and graph match; constrained generated node arguments to exact entity hints or dependency outputs, with regression tests. Investigated q26's ambiguity and added clarification and unweighted-path tests.
- **Syed Sufiyan (NS26Z168):** Built and integrated the manifest-driven orchestrator: graph catalogue and entity hints, structured query interpretation and planning, multi-tool subgraph chaining, evidence-bound answers and status handling, retry/cache and checkpointed tracing, the batch runner, offline public scoring, and regression tests. Ran and analysed repeated public-set evaluations.
- **Mohd Hamza (PH22B016):** Code review and independent reference checks, failure analysis, registry bug investigation and clean-setup review. Wrote this report.

AI-use acknowledgment. AI-assisted coding was used. The group remains responsible for validating its outputs and explaining the submitted implementation.